

Programación declarativa

- Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

5. Ajuste de patrones

- En Haskell:
 - la definición de una función consta de una o más ecuaciones;
 - la parte izquierda de cada ecuación tiene el nombre de la función seguido de uno o más patrones, uno por argumento.
- Cuando tiene que evaluar una función aplicada a un argumento, Haskell necesita ver cómo el valor se ajusta al patrón. A ese proceso se le llama *ajuste* (o *encaje*) de patrones.

Ejemplo

```
fn, fn' :: ((Int, Char), Int, Bool) -> Int
fn (p, n, _) = fst p + n
fn' ((i, _), n, b) = i + n
```

- Quiero evaluar `fn ((22, 'a'), 33, True)`:
 - Hay que ajustar el valor `((22, 'a'), 33, True)` al patrón `(p, n, _)`:
 - $p \mapsto (22, 'a')$, $n \mapsto 33$ y `True` se desprecia.
 - El lado derecho `fst p + n` se convierte en `fst (22, 'a') + 33`.
- Quiero evaluar `fn' ((22, 'a'), 33, True)`:
 - Hay que ajustar el valor `((22, 'a'), 33, True)` al patrón `((i, _), n, b)`:
 - Primero, hay que ajustar `(22, 'a')` al patrón `(i, _)`:
 - * $i \mapsto 22$ y la `'a'` se desprecia.
 - $n \mapsto 33$, $b \mapsto \text{True}$.
 - El lado derecho `i + n` se convierte en `22 + 33`.

El orden importa

- Cuando una función tiene varias ecuaciones, se prueban **en orden** y se usa la primera cuyo patrón encaja:

```
coger :: Int -> [a] -> [a]
coger 0 _ = []
```

```

| coger _ [] = []
| coger n (x:xs) = x : coger (n-1) xs

```

- Al evaluar `coger 2 [7, 8, 9]`:
 - `coger 0 _`: el valor `2` no encaja con el literal `0`. Falla.
 - `coger _ []`: el valor `2` encaja con el patrón `_`, pero `[7, 8, 9]` no encaja con `[]`. Falla.
 - `coger n (x:xs)`: $n \mapsto 2$, $x \mapsto 7$, $xs \mapsto [8, 9]$. Encaja.
- Así que `coger 2 [7, 8, 9]  $\rightsquigarrow$  7 : coger 1 [8, 9]`.
- Si ninguna ecuación encaja, la evaluación falla con un error como este: `*** Exception: Non-exhaustive patterns in function coger`

Patrones

- Los patrones que conocemos hasta ahora se construyen con estas reglas:

patrón	valores que se ajustan
variable <code>x</code>	cualquier valor; <code>x</code> toma ese valor
<code>_</code>	cualquier valor; no se guarda nada
literal (como <code>0</code> , <code>'a'</code>)	exactamente ese valor
$(p_1, \dots, p_n)$	una tupla $(v_1, \dots, v_n)$ donde cada v_i se ajusta a su p_i
<code>[]</code>	la lista vacía
$p_1 : p_2$	una lista no vacía cuya cabeza se ajusta a p_1 y su resto a p_2
$[p_1, \dots, p_n]$	abreviatura de $p_1 : \dots : p_n : []$

- Más en general, representando por `C` un constructor cualquiera:

patrón	valores que se ajustan
variable <code>x</code>	cualquier valor; <code>x</code> toma ese valor
<code>_</code>	cualquier valor; no se guarda nada
<code>C p₁ ... p_n</code>	<code>C v₁ ... v_n</code> donde cada v_i se ajusta a su p_i

- Un literal es un constructor sin argumentos.
- Los patrones se pueden anidar: `((i, _), n, _)`, `(x, y):_`, `[(0, y)]`.
- En Haskell, una variable solo puede aparecer una vez en un patrón:
 - iguales `(x, x) = True` es un error.
- Cosas que **no** son patrones: `x + 1`, `xs ++ [x]`, `fst p`, `2 * n`.
 - En un patrón no puede haber operadores definidos:
 - solo constructores (y valores literales) y variables (y `_`).

Otro ejemplo

```

| f :: [(Int, Int)] -> Int
| f [] = 0
-- ecuación 1

```

```

f ((0, y):_) = y           -- ecuación 2
f [(x, y)] = x * y         -- ecuación 3
f ((x, y):(x', y'):_) = x + y + x' + y' -- ecuación 4

```

- Algunos encajes:

valor	ecuación	sustitución	resultado
[]	1	vacía	0
[(1, 2)]	3	x=1, y=2	2
[(1, 2), (3, 4)]	4	x=1, y=2, x'=3, y'=4	10
[(0, 5)]	2	y=5	5
[(0, 1), (2, 3), (4, 5)]	2	y=1	1

- [(0, 5)] encaja con las ecuaciones 2 y 3; [(0, 5), (2, 3)] con las ecuaciones 2 y 4.
- Haskell no elige la ecuación *más específica* ni la *mejor*: usa la primera que encaja.

El algoritmo

- Demos por hecho que el patrón es correcto: solo usa constructores, variables y valores literales y no usa la misma variable varias veces.

Sustitución = [(Variable, Valor)]

$ajusta : (\text{Patrón}, \text{Valor}) \rightarrow \text{Sustitución} \cup \{\text{FALLO}\}$

$ajustaLista : [(\text{Patrón}, \text{Valor})] \rightarrow \text{Sustitución} \cup \{\text{FALLO}\}$

$combina : [\text{Sustitución} \cup \{\text{FALLO}\}] \rightarrow \text{Sustitución} \cup \{\text{FALLO}\}$

$ajustaLista \text{ tareas} = combina (\text{map } ajusta \text{ tareas})$

combina concatena las sustituciones; si alguna es FALLO, el resultado es FALLO.

$ajusta (p, v) =$

según la forma de p :

variable x : [(x , v)]

— : []

$C p_1 \dots p_n$: **si** $v = C v_1 \dots v_n$, con el mismo constructor C

entonces $ajustaLista [(p_1, v_1), \dots, (p_n, v_n)]$

si no FALLO

- En particular, para los constructores que conocemos hasta ahora, tuplas y listas:

```

ajusta (p, v) =
  según la forma de p:
    variable x      : [(x, v)]
    —              : []
    literal k       : si v = k entonces [] si no FALLO
    (p1, ..., pn)  : si v = (v1, ..., vn)
                      entonces ajustaLista [(p1, v1), ..., (pn, vn)]
                      si no FALLO
    []              : si v = [] entonces [] si no FALLO
    p1:p2         : si v = v1:v2
                      entonces ajustaLista [(p1, v1), (p2, v2)]
                      si no FALLO

```

El ejemplo desarrollado con el algoritmo

Evaluamos $f \ [(0, 5), (2, 3)]$ siguiendo el pseudocódigo. Intentamos el ajuste con el patrón de la primera ecuación, $[]$:

```

ajusta ([], [(0, 5), (2, 3)])
= FALLO

```

– constructores distintos: $[]$ frente a $:$

Como es FALLO, probamos con el patrón de la siguiente ecuación, $(0, y) : _$.

```

ajusta ((0, y):_, [(0, 5), (2, 3)])
= ajustaLista [(0, y), (0, 5)], (_, [(2, 3)])
= combina [ajusta ((0, y), (0, 5)), ajusta (_, [(2, 3)])]
= combina [ajustaLista [(0, 0), (y, 5)], []]
= combina [combina [ajusta (0, 0), ajusta (y, 5)], []]
= combina [combina [], [(y, 5)], []]
= combina [(y, 5)], []
= [(y, 5)]

```

– mismo constructor $:$
– mismo constructor $(,)$; caso $_$
– mismo constructor 0 , sin argumentos; variable

- La ecuación 2 encaja, así que no probamos más.
- Haskell usa la ecuación 2 con $y \mapsto 5$: $f \ [(0, 5), (2, 3)] \rightsquigarrow 5$.
- El patrón de la ecuación 4 también encajaría, pero nunca llega a probarse.

Ejercicios

- Desarrolla con el algoritmo los ajustes:
 - $(\text{True}, [5])$ con (True, x) ;
 - $(\text{True}, 5)$ con (False, x) ;
 - $(\text{True}, 5)$ con $(\text{True}, [x])$;
 - $[[1], [2]]$ con $[x, [y]]$.

- Desarrolla el ajuste del valor $[(0, 5), (2, 3)]$ a los patrones de las ecuaciones 3 y 4 de f , es decir, a $[(x, y)]$ y a $(x, y):(x', y'):_$.
- Completa las cabeceras y las definiciones de estas funciones.

```
sumaCosas, sumaCosas' :: ... -> Int
sumaCosas ... = x + y + z + u + v
sumaCosas' t = ...

suma, suma' :: Int
suma  = sumaCosas  ((1, (2, 5)), (4, 1))  -- suma  = 1+2+5+4+1 = 13
suma' = sumaCosas' ((1, (2, 5)), (4, 1))  -- suma' = 1+2+5+4+1 = 13
```

- Hay un truco para pedir a Haskell (a `ghci`, concretamente) que nos muestre el resultado del ajuste de un valor a un patrón. Por ejemplo, mira esta función:

```
| dameSust (p:(x', y'):_ ) = (p, x', y')
```

Si ejecutas `dameSust [(0, 5), (2, 3)]`, `ghci` muestra como resultado $((0, 5), 2, 3)$. Y, si ejecutas `dameSust [(1, 5)]`, muestra un mensaje de error. Define funciones de este estilo para algunas de tus soluciones de arriba y comprueba que los resultados son como tú creías. Prueba, en particular, a ajustar $(\text{True}, 5)$ al patrón $(\text{True}, [x])$ y trata de interpretar qué pasa.